# PROYECTO CRUD ANGULAR

**José Carlos Mora**

**23/02/2026**

**2ºDAM**

**Proyecto crud angular**

**Índice**

**1. Introducción y Contextualización del Proyecto.....3**
**2. Desarrollo.....4**
**3. Conclusión.....6**
**4. Bibliografia, Webgrafia e IA.....7**

## 1. Introducción y Contextualización del Proyecto

### ¿Qué es CoachX?

CoachX es una aplicación móvil Android diseñada para digitalizar y centralizar la gestión de clubes de fútbol base. Integra en una única plataforma todas las herramientas que actualmente los clubes manejan de forma dispersa: WhatsApp, hojas de cálculo, pizarras físicas y documentos en papel.

La aplicación se estructura en cinco roles diferenciados — Coordinador, Entrenador, Delegado, Jugador y Aficionado (cada uno con permisos y funcionalidades adaptadas a su responsabilidad dentro del club). Además, incorpora Inteligencia Artificial para la generación de entrenamientos personalizados y el análisis del rendimiento de los jugadores.

### 1.2 ¿Por qué CoachX? Justificación

El fútbol base español mueve cifras muy relevantes: en 2025 se registraron más de 4,3 millones de licencias federadas, cerca de 21.000 clubes activos y entre 60.000 y 70.000 entrenadores en todo el país. A pesar de esta dimensión, la gran mayoría de los clubes de base carecen de herramientas digitales adecuadas para su gestión.

La realidad cotidiana de un coordinador o entrenador de fútbol base es trabajar con múltiples aplicaciones sin conexión entre sí: grupos de WhatsApp para avisos, Excel para el control de asistencias, notas en papel para las alineaciones y carpetas físicas para los datos de los jugadores. Este escenario genera ineficiencias, pérdidas de información y una carga administrativa innecesaria que resta tiempo y energía a la parte técnica y formativa.

CoachX nace para resolver exactamente ese problema: una sola plataforma que lo gestiona todo, accesible desde el móvil, con diseño intuitivo pensado también para usuarios con poca experiencia digital, y con IA integrada que actúa como asistente técnico del entrenador.

### 1.2 Qué es CoachX Angular

CoachX Angular es la version web de la plataforma CoachX. Se trata de una aplicación de gestión de jugadores de fútbol base que implementa un CRUD completo: crear, leer, actualizar y eliminar jugadores desde el navegador, con los datos persistidos en JSON Server.

## 2. Desarrollo

### 2.1 Desarrollo Back-end

El back-end de CoachX Angular se implementa con JSON Server, una herramienta Node.js que convierte un archivo db.json en una API REST completamente funcional. Soporta los métodos GET, POST, PUT y DELETE de forma automática, lo que permite desarrollar y probar el front-end sin necesidad de un servidor real.

El servidor corre en el puerto 3000 con el comando npx json-server db.json --port 3000, de forma paralela al servidor de Angular que corre en el puerto 4200. Ambos tienen que estar activos al mismo tiempo para que la aplicación funcione correctamente.

#### 2.1.1 Modelo de datos — db.json

El archivo db.json se crea manualmente en la raíz del proyecto. Contiene un array de jugadores con los campos necesarios para la gestión del club:

El campo id es gestionado automáticamente por JSON Server: al hacer un POST sin id, el servidor asigna el siguiente número disponible. El campo estado puede tomar los valores Disponible, Lesionado o Sancionado, lo que permite reflejar la situación real del jugador en la plantilla.

En mi caso, lo creé manualmente


```
    {
      "id": "1",
      "nombre": "Carlos García",
      "dorsal": 9,
      "posicion": "Delantero",
      "equipo": "Juvenil A",
      "estado": "Disponible",
      "foto": "https://ui-avatars.com/api/?name=Carlos+Garcia&background=1B2A4A&color=fff&size=200"
    },
    {
      "id": "2",
      "nombre": "Alejandro Ruiz",
      "dorsal": 1,
      "posicion": "Portero",
      "equipo": "Juvenil A",
      "estado": "Disponible",
      "foto": "https://ui-avatars.com/api/?name=Alejandro+Ruiz&background=E8470A&color=fff&size=200"
    },
    {
      "id": "3",
      "nombre": "Mario Sánchez",
      "dorsal": 4,
      "posicion": "Defensa",
      "equipo": "Cadete B",
      "estado": "Lesionado",
      "foto": "https://ui-avatars.com/api/?name=Mario+Sanchez&background=1B2A4A&color=fff&size=200"
    },
    {
      "id": "4",
      "nombre": "Pablo Torres",
      "dorsal": 8,
      "posicion": "Centrocampista",
      "equipo": "Juvenil A",
      "estado": "Disponible",
      "foto": "https://ui-avatars.com/api/?name=Pablo+Torres&background=E8470A&color=fff&size=200"
    },
```

## 2.2 Desarrollo Front-end

Cada componente importa únicamente los módulos que necesita, el servicio centraliza todas las llamadas HTTP y el sistema de rutas conecta las URLs con los componentes correspondientes.

```
OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

M\Optativa> ng new coachx-angular
ylesheet system would you like to use? CSS            [ https://developer.mozilla.org/docs/Web/CSS                    ]
ant to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? No
 tools do you want to configure with Angular best practices? https://angular.dev/ai/develop-with-ai

 tools do you want to configure with Angular best practices? https://angular.dev/ai/develop-with-ai None
chx-angular/angular.json (2000 bytes)
```

```
PROBLEMS 7    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\2 DAM\Optativa\coachx-angular> ng generate component create-jugador
CREATE src/app/create-jugador/create-jugador.spec.ts (604 bytes)
CREATE src/app/create-jugador/create-jugador.ts (228 bytes)
CREATE src/app/create-jugador/create-jugador.css (0 bytes)
CREATE src/app/create-jugador/create-jugador.html (30 bytes)
```

```
PS D:\2 DAM\Optativa\coachx-angular> ng generate component details-jugador
CREATE src/app/details-jugador/details-jugador.spec.ts (611 bytes)
CREATE src/app/details-jugador/details-jugador.ts (232 bytes)
CREATE src/app/details-jugador/details-jugador.css (0 bytes)
CREATE src/app/details-jugador/details-jugador.html (31 bytes)
```

```
PS D:\2 DAM\Optativa\coachx-angular> ng generate component edit-jugador
CREATE src/app/edit-jugador/edit-jugador.spec.ts (590 bytes)
CREATE src/app/edit-jugador/edit-jugador.ts (220 bytes)
CREATE src/app/edit-jugador/edit-jugador.css (0 bytes)
CREATE src/app/edit-jugador/edit-jugador.html (28 bytes)
```

```
PS D:\2 DAM\Optativa\coachx-angular> ng generate component jugador-card
CREATE src/app/jugador-card/jugador-card.spec.ts (590 bytes)
CREATE src/app/jugador-card/jugador-card.ts (220 bytes)
CREATE src/app/jugador-card/jugador-card.css (0 bytes)
CREATE src/app/jugador-card/jugador-card.html (28 bytes)
```

```
PS D:\2 DAM\Optativa\coachx-angular> ng generate component jugadores
CREATE src/app/jugadores/jugadores.spec.ts (575 bytes)
CREATE src/app/jugadores/jugadores.ts (209 bytes)
CREATE src/app/jugadores/jugadores.css (0 bytes)
CREATE src/app/jugadores/jugadores.html (25 bytes)
```

```
PS D:\2 DAM\Optativa\coachx-angular> ng g service jugador
CREATE src/app/jugador.spec.ts (342 bytes)
CREATE src/app/jugador.ts (120 bytes)
PS D:\2 DAM\Optativa\coachx-angular> ng g interface jugadoresInterface
CREATE src/app/jugadores-interface.ts (42 bytes)
```

### 2.2.1 Estructura del proyecto y comandos de instalación

La estructura de carpetas dentro de src/app/ es la siguiente:

- **src/app/**
- **jugadoresinterface.ts**

- **jugadorservice.ts**
- **app.routes.ts**
- **app.html**
- **jugadores/**
- **jugador-card/**
- **create-jugador/**
- **details-jugador/**
- **edit-jugador/**

### 2.2.2 Interface y Servicio

La interface define la estructura de datos del jugador y el servicio centraliza todas las llamadas a la API

Mora_JC_OPT > Mora_JC_OPT > coachx-angular > src > app > TS jugadoresinterface.ts > Jugadoresinterface

```typescript
export interface Jugadoresinterface {
  id?: string;
  nombre: string;
  dorsal: number;
  posicion: string;
  equipo: string;
  estado: string;
  foto: string;
}
```

```typescript
})
export class Jugadorservice {

  private apiUrl = 'http://localhost:3000/jugadores';

  async getallJugadores(): Promise<Jugadoresinterface[]> {
    try {
      const response = await fetch(this.apiUrl);
      if (!response.ok) throw new Error('Error al obtener jugadores');
      return await response.json();
    } catch (error) {
      console.error('Error en getallJugadores:', error);
      return [];
    }
  }

 async getJugadorPorNombre(nombre: string) {
   const response = await fetch(this.apiUrl);
   const jugadores = await response.json();

   return jugadores.find(
     (j: any) => j.nombre.trim().toLowerCase() === nombre.trim().toLowerCase()
   ) || null;
 }



  async createJugador(jugador: Jugadoresinterface): Promise<Jugadoresinterface | null> {
    try {
      const response = await fetch(this.apiUrl, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(jugador)
      });
      if (!response.ok) throw new Error('Error al crear jugador');
      return await response.json();
    } catch (error) {
      console.error('Error en createJugador:', error);
      return null;
```

### 2.2.3 Componente lista — Jugadores

Componente principal que carga todos los jugadores al inicializarse mediante ngOnInit y los pasa al componente hijo JugadorCard mediante @Input

```
import { Jugadoresinterface } from '../jugadoresinterface';
import { JugadorCard } from '../jugador-card/jugador-card';
import { Jugadorservice } from '../jugadorservice';

@Component({
  selector: 'app-jugadores',
  imports: [CommonModule, JugadorCard, RouterModule],
  templateUrl: './jugadores.html',
  styleUrl: './jugadores.css',
})
export class Jugadores implements OnInit {

  jugadoreslista: Jugadoresinterface[] = [];
  filteredjugadoreslista: Jugadoresinterface[] = [];
  jugadorservice: Jugadorservice = inject(Jugadorservice);
  private cdr = inject(ChangeDetectorRef);

  constructor() {

  }

  ngOnInit(): void {
    this.jugadorservice.getallJugadores()
      .then((datos: Jugadoresinterface[]) => {
        this.jugadoreslista = datos;
        this.filteredjugadoreslista = datos;
        this.cdr.detectChanges();
      })
      .catch(error => {
        console.error('Error al cargar jugadores:', error);
      });
  }

  onsearch(busqueda: string) {
    if (!busqueda) {
      this.filteredjugadoreslista = this.jugadoreslista;
      return;
    }
```

### 2.2.4 Componente tarjeta hijo — JugadorCard

Recibe un jugador mediante @Input y muestra su foto, dorsal, nombre, posicion, equipo y estado con un color según disponibilidad

```
import { Component, Input } from '@angular/core';
import { RouterModule } from '@angular/router';
import { CommonModule } from '@angular/common';
import { Jugadoresinterface } from '../jugadoresinterface';

@Component({
  selector: 'app-jugador-card',
  imports: [RouterModule, CommonModule],
  templateUrl: './jugador-card.html',
  styleUrl: './jugador-card.css',
})
export class JugadorCard {

  @Input() jugador!: Jugadoresinterface;

  getEstadoClass(estado: string): string {
    switch (estado) {
      case 'Disponible': return 'estado-disponible';
      case 'Lesionado':  return 'estado-lesionado';
      case 'Sancionado': return 'estado-sancionado';
      default:           return '';
    }
  }
}
```

### 2.2.5 Componente crear — CreateJugador

Utiliza FormGroup con FormControl y Validators.required para cada campo. Al enviar construye el objeto jugador, llama al servicio y navega a la lista si el resultado es correcto

Mora_JC_OPT > Mora_JC_OPT > coachx-angular > src > app > create-jugador > TS create-jugador.ts > CreateJugador

```
import { Component, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormControl, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { Jugadorservice } from '../jugadorservice';

@Component({
  selector: 'app-create-jugador',
  imports: [CommonModule, ReactiveFormsModule],
  templateUrl: './create-jugador.html',
  styleUrl: './create-jugador.css',
})
export class CreateJugador {

  jugadorservice = inject(Jugadorservice);
  router         = inject(Router);
  isSubmitting   = false;

  formJugador = new FormGroup({
    nombre:   new FormControl('', [Validators.required]),
    dorsal:   new FormControl('', [Validators.required]),
    posicion: new FormControl('', [Validators.required]),
    equipo:   new FormControl('', [Validators.required]),
    estado:   new FormControl('Disponible', [Validators.required]),
    foto:     new FormControl('', [Validators.required])
  });

  async onSubmit() {
    if (this.formJugador.valid && !this.isSubmitting) {
      this.isSubmitting = true;

      const nuevoJugador = {
        nombre:   this.formJugador.value.nombre?.trim()   ?? '',
        dorsal:   Number(this.formJugador.value.dorsal),
        posicion: this.formJugador.value.posicion?.trim() ?? '',
        equipo:   this.formJugador.value.equipo?.trim()   ?? '',
        estado:   this.formJugador.value.estado           ?? 'Disponible',
        foto:     this.formJugador.value.foto?.trim()     ?? ''
```

### 2.2.6 Componente detalle — Details

Obtiene el nombre del jugador desde los parámetros de la ruta con ActivatedRoute, lo busca en la API y muestra su ficha completa. Incluye botones para editar y eliminar directamente desde el detalle

Mora_JC_OPT > Mora_JC_OPT > coachx-angular > src > app > details-jugador > TS details-jugador.ts > Details

```
import { Component, inject, OnInit, ChangeDetectorRef } from '@angular/core';
import { ActivatedRoute, Router } from '@angular/router';
import { CommonModule } from '@angular/common';
import { Jugadorservice } from '../jugadorservice';
import { Jugadoresinterface } from '../jugadoresinterface';

@Component({
  selector: 'app-details-jugador',
  imports: [CommonModule],
  templateUrl: './details-jugador.html',
  styleUrl: './details-jugador.css',
})
export class Details implements OnInit {

  route:          ActivatedRoute = inject(ActivatedRoute);
  router:         Router         = inject(Router);
  jugadorservice                 = inject(Jugadorservice);
  jugador:        Jugadoresinterface | undefined;
  private cdr = inject(ChangeDetectorRef);

  constructor() {

  }

  ngOnInit(): void {
    const jugadornombre = this.route.snapshot.params['nombre'];

    console.log('Buscando jugador:', jugadornombre);

    this.jugadorservice.getJugadorPorNombre(jugadornombre)
      .then((jugador) => {
        this.jugador = jugador;
        console.log('Jugador cargado:', jugador);
        this.cdr.detectChanges();
      })
      .catch(error => {
        console.error('Error al cargar jugador:', error);
      });
```

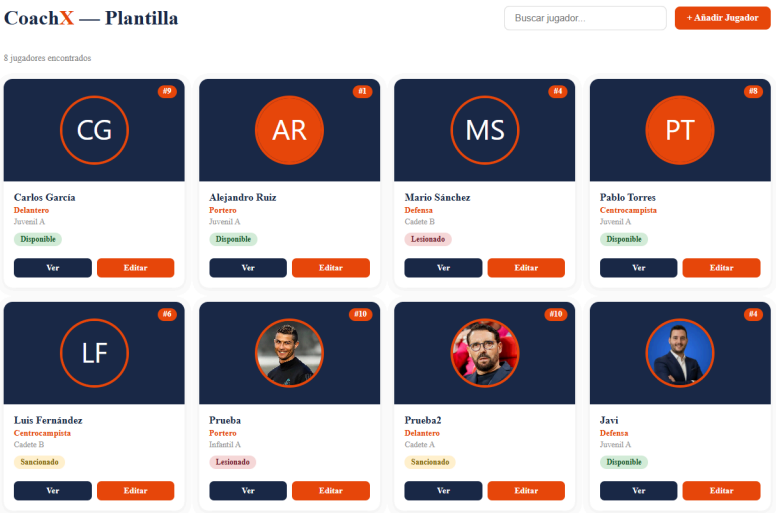
### 2.2.7 Componente editar — EditJugador

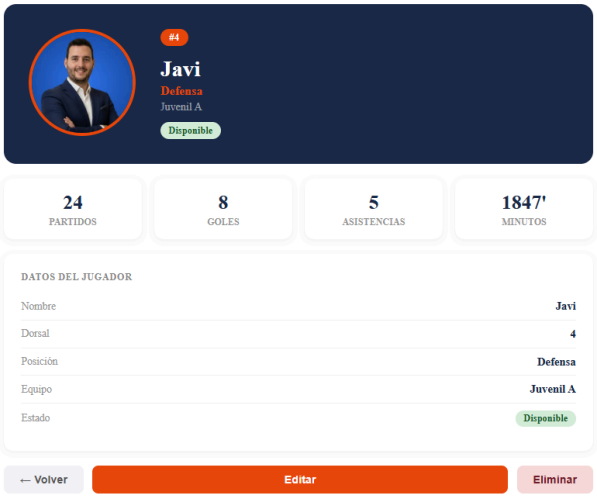
Al inicializarse carga los datos actuales del jugador mediante patchValue para pre-rellenar el formulario. Al guardar llama a updateJugador y navega al detalle del jugador actualizado

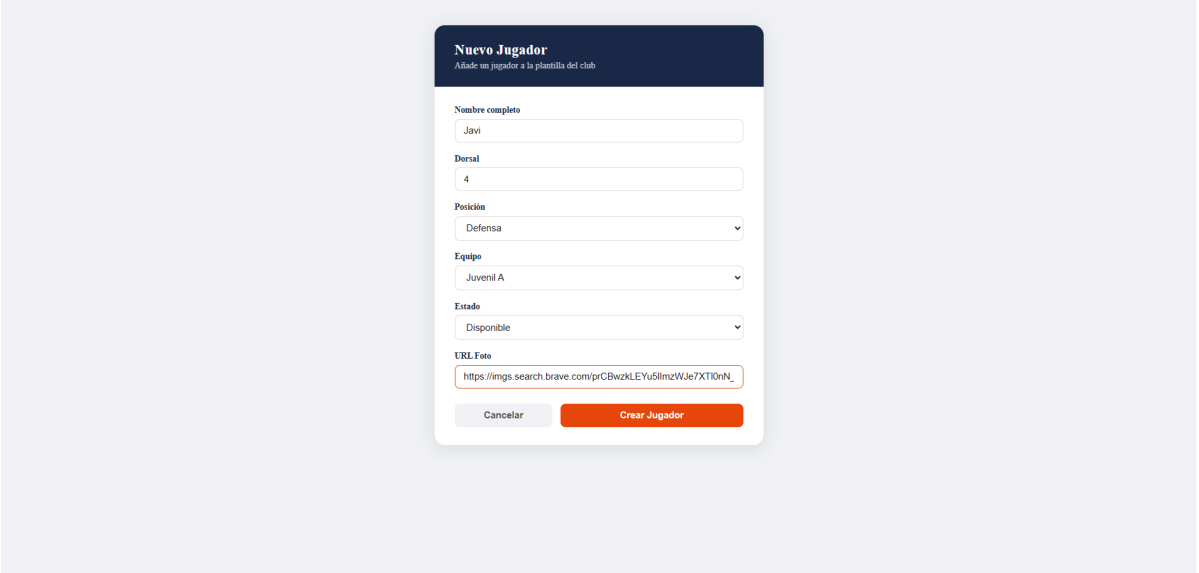
Mora_JC_OPT > Mora_JC_OPT > coachx-angular > src > app > edit-jugador > TS edit-jugador.ts > EditJugador > cargarJugador

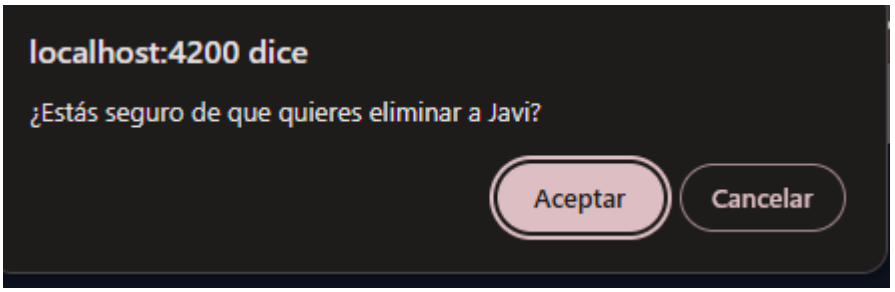
```angular_core
import { Component, inject, OnInit, ChangeDetectorRef } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormControl, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router, ActivatedRoute } from '@angular/router';
import { Jugadorservice } from '../jugadorservice';

@Component({
  selector: 'app-edit-jugador',
  imports: [CommonModule, ReactiveFormsModule],
  templateUrl: './edit-jugador.html',
  styleUrl: './edit-jugador.css',
})
export class EditJugador implements OnInit {

  jugadorservice = inject(Jugadorservice);
  router = inject(Router);
  route = inject(ActivatedRoute);
  cd = inject(ChangeDetectorRef);

  jugadorNombre = '';
  isSubmitting = false;
  isLoading = true;

  formJugador = new FormGroup({
    nombre: new FormControl('', [Validators.required]),
    dorsal: new FormControl('', [Validators.required]),
    posicion: new FormControl('', [Validators.required]),
    equipo: new FormControl('', [Validators.required]),
    estado: new FormControl('', [Validators.required]),
    foto: new FormControl('', [Validators.required])
  });

  ngOnInit(): void {
    const nombreParam = this.route.snapshot.params['nombre'];

    if (!nombreParam) {
      this.router.navigate(['/']);
      return;
    }
```

## Capturas Finales






## 3. Conclusión

El desarrollo de CoachX en Angular ha permitido aplicar de forma directa el patrón aprendido previamente: interfaz de datos, servicio centralizado para las peticiones, componente lista que carga en ngOnInit, componente

hijo con @Input y formularios reactivos tanto para crear como para editar mediante patchValue.

A nivel técnico, se ha implementado un CRUD completo sobre la entidad Jugador, un buscador en tiempo real sin nuevas llamadas a la API, badges de estado con estilos dinámicos según disponibilidad y navegación fluida entre vistas gracias al router de Angular

## 4. Bibliografia, Webgrafia e IA

- Documentación oficial
- Apuntes de clase
- Claude.ai para los css (le he dicho como los quería y me los ha hecho) a partir de ahí, lo he adaptado a mi gusto, además, lo he usado para un fallo que tuve y que no podía solucionar, ya que cuando le daba a editar se quedaba en cargando y no entendía el por qué

```
export class EditJugador implements OnInit {



  jugadorservice = inject(Jugadorservice);

  router = inject(Router);

  route = inject(ActivatedRoute);

  cd = inject(ChangeDetectorRef);
```

Me hizo esto concretamente